

115學年度第1學期資料結構作業一

動態環狀佇列：實作、驗證與效能觀察

項目	說明
作業比重	學期總成績 10%
作業形式	個人作業
截止日期	2026/10/02（五） 23:59
繳交方式	課程平台上傳一個壓縮檔
使用語言	C++20

一、作業目標

本作業的主題是實作一個可自動擴充容量的環狀陣列佇列，並與每次刪除元素時都搬移資料的佇列實作進行比較。

完成本作業後，你應能：

- 區分佇列抽象資料型態（Queue ADT）與其底層實作；
- 維護 `front`、`rear`、`size` 與 `capacity` 之間的不變條件（invariants）；
- 正確處理環狀索引、容量擴充、空佇列及動態記憶體；
- 使用邊界情況與差異測試驗證程式；
- 根據程式的實際行為，估計基本操作與資料搬移的次數；
- 以小型實驗比較理論上的操作次數與實際執行時間。

二、起始檔案

教師將提供：

```
starter/  
├─ circular_queue.hpp  
├─ circular_queue.cpp  
├─ naive_shift_queue.hpp  
├─ public_tests.cpp  
├─ tests.cpp  
├─ benchmark.cpp  
├─ plot_results.py  
├─ answer_sheet.md  
├─ ai_disclosure.md  
└─ README.md
```

- `circular_queue.hpp` 已定義公開介面。

- `circular_queue.cpp` 內含需要完成的區段。
- `naive_shift_queue.hpp` 是結果正確但效率較低的比較版本，不得修改。
- `benchmark.cpp` 與 `plot_results.py` 已完成實驗及繪圖的主要流程。你只需編譯、執行並保留結果，不必自行設計整套實驗程式。
- `public_tests.cpp` 用來檢查基本功能；完整評分另包含隱藏測試。
- `tests.cpp`、`answer_sheet.md`、`ai_disclosure.md` 是需要你完成的骨架；`README.md` 已列出所有指令，你只需補上實際使用的執行環境。

關於開發環境

本作業不指定開發環境，也不提供統一的虛擬機或容器。你可以使用任何作業系統與編譯器。讓自己的程式能在別人的機器上編譯、執行並通過檢查，是本作業的一部分。

評分時使用的指令與工具全部列在 `README.md` 第二至五節，沒有隱藏條件。如果你的程式只能在你自己的機器上跑，那是需要修正的問題，不是評分的例外。

其中記憶體安全這一項要特別注意：AddressSanitizer 的記憶體洩漏偵測（LeakSanitizer）在 Linux 上預設開啟，但 **macOS 不支援**。若你在 macOS 開發，忘記在解構函式釋放記憶體時，本機不會給你任何錯誤訊息，因此繳交前請務必找一台 Linux 機器或容器重跑一次 `README.md` 第四節的檢查。

三、實作 `CircularQueue` (50%)

(一) 公開介面

不得修改下列公開介面：

```
class CircularQueue {
public:
    explicit CircularQueue(std::size_t initial_capacity = 4);
    ~CircularQueue();

    CircularQueue(const CircularQueue&) = delete;
    CircularQueue& operator=(const CircularQueue&) = delete;

    bool empty() const;
    std::size_t size() const;
    std::size_t capacity() const;
    int front() const;
    void enqueue(int value);
    void dequeue();

private:
    int* data_;
    std::size_t capacity_;
    std::size_t size_;
    std::size_t front_;
    std::size_t rear_;
```

```
void grow();  
};
```

`grow()` 是起始檔案中已提供的私有輔助函式；你可以使用它，但不得新增或修改任何 `public` 成員。

(二) 實作規格

- 必須使用 `new[]` 配置動態陣列，並在解構函式中正確釋放。
- `rear_` 表示下一個元素的插入位置。
- 所有索引更新都必須支援環狀回繞。
- 除了擴充容量以外，`enqueue()` 與 `dequeue()` 不得搬移原有元素。
- 佇列已滿時，將容量擴充為原本的兩倍。
- 擴充容量時，必須依照佇列中的邏輯順序複製元素。完成重排後，令 `front_ = 0`、`rear_ = size_`，再完成本次插入。
- `initial_capacity == 0` 時，丟出 `std::invalid_argument`。
- 對空佇列呼叫 `front()` 或 `dequeue()` 時，丟出 `std::underflow_error`。
- 不得使用 STL 容器作為內部儲存空間。
- 只能使用 C++ 標準函式庫，不得依賴特定平台或編譯器的行為。
- 不得發生記憶體洩漏、重複釋放、陣列越界或釋放後繼續使用等錯誤。

程式執行期間應始終維持以下條件：

```
0 <= size_ <= capacity_  
0 <= front_ < capacity_  
0 <= rear_ < capacity_  
rear_ == (front_ + size_) mod capacity_  
第 i 個邏輯元素位於 (front_ + i) mod capacity_
```

當佇列為空或完全填滿時，都可能出現 `front_ == rear_`，因此必須使用 `size_` 區分這兩種狀態。

(三) 評分方式

項目	作業配分
公開測試涵蓋的基本功能	10%
隱藏測試：環狀索引、容量擴充、多次擴充、例外處理與長操作序列	30%
記憶體安全	5%
程式可讀性與規格遵循	5%

隱藏測試會使用不同的 `initial_capacity` 與操作序列，請勿只針對公開範例撰寫程式。記憶體安全以 `README.md` 第四節的 `AddressSanitizer` 與 `UndefinedBehaviorSanitizer` 檢查為準。

四、正確性驗證 (30%)

本部分要求三個測試，以及一個與測試共用操作序列的反例分析。增加大量相似測試不會獲得額外加分。

（一）三個指定測試（15%）

在 `tests.cpp` 中完成：

1. **空佇列與單一元素測試**：涵蓋空佇列的例外，以及加入、讀取、刪除一個元素後回到空佇列。
2. **環狀回繞後擴充測試**：先使佇列中的邏輯順序跨過陣列尾端，再於佇列已滿時觸發容量擴充，檢查所有元素的輸出順序。
3. **隨機差異測試**：使用固定的亂數種子，產生至少 2,000 次合法的 `enqueue()`/`dequeue()` 操作，逐步與 `std::queue<int>` 比較 `size()`、`empty()` 與 `front()`。請在 `tests.cpp` 中以具名常數記錄亂數種子，例如 `constexpr unsigned int kSeed = ...;`。

只有在佇列非空時才能產生 `dequeue()` 操作。`std::queue` 只能作為測試時的對照標準，不得用於 `CircularQueue` 的內部實作。

（二）容量擴充錯誤的最短反例（15%）

考慮以下錯誤策略：

擴充容量時，直接從陣列索引 0 開始複製 `size_` 個元素，而不是從 `front_` 開始，依照環狀順序複製。

令 `initial_capacity = 2`，在 `answer_sheet.md` 中完成：

1. 寫出一個能暴露此錯誤的最短操作序列，最多使用 5 次 `enqueue()`/`dequeue()`。
2. 以不超過 5 列的表格顯示每一步的 `front_`、`rear_`、`size_`、佇列中的邏輯順序，以及陣列中的實際排列；未使用的位置以 `_` 表示。
3. 寫出正確實作與錯誤實作在容量擴充後各自得到的元素順序。
4. 指出上述錯誤策略破壞了哪一條不變條件。

此處的操作序列應同時用於前一小節的「環狀回繞後擴充測試」，不必為同一問題另外設計一組操作。

五、小型效能實驗（20%）

比較以下兩種實作：

- 你完成的 `CircularQueue`；
- 教師提供的 `NaiveShiftQueue`，其 `dequeue()` 會將剩餘元素逐一向前搬移。

（一）實驗流程

使用三個資料規模：

`2^9`, `2^12`, `2^15`

`benchmark.cpp` 以 `CircularQueue q;` 建立受測的環狀佇列，因此初始容量為 4。

對每個 `n` 及每種佇列實作：

1. 先放入 `n` 個元素，這一階段不計時。
2. 接著計時執行 `n` 輪以下操作：

```
checksum += q.front();  
q.dequeue();  
q.enqueue(next_value);
```

3. 每組設定執行 3 次，以中位數進行比較。

主要實驗共有 $3 \times 2 \times 3 = 18$ 次計時執行。`benchmark.cpp` 會輸出 `results.csv`，`plot_results.py` 會產生圖表。

效能測試須使用 `-O2` 編譯。產生輸入、預先放入元素、讀寫檔案與螢幕輸出均不得納入計時。

不同機器的絕對執行時間本來就會有差異，評分看的是你自己數據呈現出的成長趨勢與說明，不是與其他同學的數字比較。

（二）操作次數與成長趨勢（10%）

本小節不需使用 Big-O 符號。請根據兩種佇列的實際操作方式回答：

1. 預先放入元素後、計時開始時，`CircularQueue` 的 `size()` 與 `capacity()` 各是多少？在計時的每一輪中，`dequeue()` 與 `enqueue()` 需要搬移多少個既有元素？為什麼這段計時過程不會觸發容量擴充？
2. `NaiveShiftQueue` 中原有 n 個元素時，每次 `dequeue()` 約需向前搬移多少個元素？連續執行 n 輪後，總共約需搬移多少次？
3. 當 n 增加為原本的 8 倍時，兩種實作所需的基本操作次數，理論上各約增加為幾倍？請簡要說明推導方式。

（三）實驗結果呈現與說明（10%）

繳交 `results.csv` 與 `performance.png`。圖表須包含座標名稱、時間單位及兩種實作的曲線。接著根據自己的實驗數據回答：

1. 列出兩種實作在三個資料規模下的中位執行時間，並計算相鄰資料規模之間的時間成長倍率。
2. 將實測的時間成長倍率，與上一小節根據操作次數推導出的成長倍率進行比較。
3. 實測結果是否符合操作次數所呈現的成長趨勢？若不完全一致，請提出一個合理解釋。不得刪除或修改不符合預期的數據。

本節（二）、（三）由學生自行撰寫的說明文字，合計以 500 個中文字以內為原則，不含題目敘述與表格中的純數值，不必寫滿。不要求使用 Big-O 符號。若理論上的操作次數倍率與實測時間倍率不完全一致，可以從固定成本、記憶體存取效率或編譯器最佳化等方向簡要說明，不必深入討論硬體細節。

六、繳交內容

請上傳 `studentID_HW1.zip`，解壓縮後應包含：

```
studentID_HW1/  
├─ circular_queue.hpp  
├─ circular_queue.cpp  
├─ tests.cpp  
└─ results.csv
```

```
|— performance.png
|— answer_sheet.pdf
|— README.md
|— ai_disclosure.md
```

- **answer_sheet.pdf** 最多 2 頁，內容為第四節的反例分析，以及第五節的操作次數分析與實驗結果說明。
- **README.md** 已列出編譯程式、執行測試、檢查記憶體安全及執行效能實驗的指令；請在其中補上你實際使用的作業系統與編譯器版本。
- 不必繳交完整的終端機輸出、完整的 AI 對話或額外實驗。

本作業不指定開發環境，但你必須自行確認繳交的程式滿足下列條件，因為評分就是依這些條件進行：

1. 以 C++20 撰寫，只使用標準函式庫，不依賴特定平台或編譯器的行為；
2. 能以 **README.md** 第二、三、五節的指令編譯通過，且不出現任何警告；
3. 以 **README.md** 第四節的 AddressSanitizer 與 UndefinedBehaviorSanitizer 執行時，不出現任何錯誤或記憶體洩漏回報。

若你的開發環境無法執行第 3 項檢查（例如 macOS 不支援 LeakSanitizer），請自行找一台 Linux 機器或容器重跑一次再繳交。繳交前建議把壓縮檔解到一個乾淨的目錄，從頭把所有指令跑過一遍，確認不會漏檔。

七、評分標準

項目	作業配分	換算學期總成績
CircularQueue 實作	50%	5.0 分
正確性驗證	30%	3.0 分
小型效能實驗	20%	2.0 分
合計	100%	10 分

若程式無法以指定指令編譯，實作部分以及依賴該實作的測試與實驗項目無法得分；可以獨立檢查的反例分析與操作次數分析仍依評分標準計分。此處的「指定指令」即 **README.md** 第二至五節所列的指令，內容與起始檔案中提供的版本相同。

八、生成式 AI 與口頭查核

本作業允許使用生成式 AI，但不以使用 AI 為前提。所有要求均可依據課堂教材與 C++ 基礎獨立完成。

- 不得交換或抄錄其他同學的程式、測試、反例或答案。
- 若未使用生成式 AI，在 **ai_disclosure.md** 勾選「未使用」即可。
- 若有使用，請填寫工具名稱、使用目的，以及你如何驗證產生的內容；以 150 個中文字為原則，不必附上完整對話。
- 使用或不使用 AI 本身不影響分數；評分依據為程式的實際行為與所繳內容。

作業繳交後，助教將隨機抽選 10%–15% 的學生進行約 5 分鐘的口試。口試不另計分，而是本次作業的通過性查核，結果只有「通過」與「未通過」兩種：

- 問題只會與本人繳交的程式、測試、反例及分析有關。
 - 可能要求說明某段程式、追蹤一小段操作、解釋測試目的，或說明操作次數與成長趨勢。
 - 能合理說明所繳內容的主要邏輯與判斷，即視為通過；不以口語是否流暢或是否能逐字背出程式為標準。
 - 口試通過者保留原作業分數；口試未通過者，本次作業成績以 0 分計算。
 - 被抽中但無故未參加者，視同未通過。因正式公假或其他經學校認可的理由無法參加者，將安排補行口試。
-

九、提交前檢查

- ☐ `CircularQueue` 通過公開測試與自己撰寫的三個指定測試。
- ☐ 「環狀回繞後擴充測試」確實先發生索引回繞，再觸發容量擴充。
- ☐ 隨機差異測試至少執行 2,000 次操作，並在 `tests.cpp` 中以具名常數記錄亂數種子。
- ☐ 已在支援 LeakSanitizer 的環境（Linux）中，以 AddressSanitizer 與 UndefinedBehaviorSanitizer 執行測試，且沒有錯誤或記憶體洩漏回報。
- ☐ 所有編譯指令都沒有出現警告。
- ☐ `results.csv` 保留 18 次原始測量結果。
- ☐ `performance.png` 具有座標名稱、單位與兩條曲線。
- ☐ `answer_sheet.pdf` 不超過 2 頁。
- ☐ `README.md` 已補上實際使用的作業系統與編譯器版本，且其中的指令可以重現結果。
- ☐ `ai_disclosure.md` 已如實填寫。
- ☐ 能說明自己所繳交的程式、測試與分析。